

Daniel Gapper

11758227

PA 2

I started by defining the `_process` struct to represent each process, including fields like arrival time (`A`), CPU burst upper bound (`B`), total CPU time required (`C`), and I/O multiplier (`M`). I added fields for the process ID, status, finishing time, CPU time run, I/O blocked time, waiting time, CPU burst, I/O burst, and flags like `originalA`, `originalC`, and `finished` to keep track of original values and completion status.

Next, I declared global variables such as `CURRENT_CYCLE`, `TOTAL_CREATED_PROCESSES`, and `TOTAL_FINISHED_PROCESSES` to monitor the simulation state. I also set constants for the random numbers file and seed value.

I implemented the `getRandNumFromFile` function to read random numbers from the "random-numbers" file, which I used in `randomOS` to generate CPU burst times using the formula `1 + (random_number % upper_bound)`.

For output, I wrote helper functions like `printStart`, `printFinal`, and `printProcessSpecifics` to display the processes and their details. The `printSummaryData` function calculates metrics like finishing time, CPU utilization, I/O utilization, throughput, average turnaround time, and average waiting time.

I included a `sortProcessesByArrival` function to sort processes based on their arrival time, which is essential for scheduling algorithms like FCFS. To read and initialize the processes from the input file, I wrote `parseInputFile`, and to reset processes between scheduler runs, I created `resetProcesses`.

For the scheduling algorithms, I started with `scheduleFCFS` for First Come First Serve scheduling. It simulates each time cycle, handling arrivals, I/O bursts, running processes, and selecting the next process based on arrival time. I made sure to update waiting times and advance `current_time` appropriately.

I then adapted this logic for the Round Robin scheduler in `scheduleRR`, incorporating time slicing. For Shortest Job First scheduling, I wrote `scheduleSJF`, selecting processes with the shortest total CPU time required, using `originalC` for comparison.

In the `main` function, I tied everything together. After parsing the input file, I ran the FCFS scheduler, reset the processes, and then ran the RR and SJF schedulers, resetting between each run. I ensured command-line arguments are handled correctly and that the program outputs the expected results.

Overall, I focused on structuring the code clearly, managing process states accurately, and ensuring time calculations are correct to simulate the scheduling algorithms effectively.

Thank you!